

Лабораторная работа №20. Работа с DOM и событиями в JavaScript

Цель лабораторной работы:

- Научиться работать с DOM-деревом;
- Освоить поиск HTML-элементов из JavaScript;
- Научиться изменять содержимое страницы;
- Освоить обработку событий пользователя;
- Создать простой TODO-лист

Шаг 1. Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте встроенный терминал
3. Перейдите в каталог с документами
4. Создайте директорию для лабораторной работы:
Lab20_DOM_Events_FIO (где FIO — ваша фамилия)
5. Перейдите в созданную директорию
6. Создайте структуру проекта:
 - **README.md**
 - **index.html**
 - **style.css**
 - **main.js**
7. Откройте созданную папку в **Visual Studio Code**

Инициализация Git-репозитория

1. Инициализируйте Git-репозиторий:
git init
2. Добавьте файлы в индекс:
git add .
3. Создайте первый коммит:

git commit -m "Initial commit"

Создание удалённого репозитория

1. Создайте пустой **public-репозиторий** на GitHub с именем:
Lab20_DOMEvents
2. Привяжите локальный репозиторий к удалённому:

git branch -M main

git remote add origin <URL_репозитория>

git push -u origin main

3. Сделайте **скриншот терминала** с выполненными командами:

- **commit**
- **remote add**
- **push**

4. Создайте папку **img** и сохраните в ней изображение удачного пуша под именем:

gitPushLab20_DOM_Events_FIO.png

5. Добавьте изменения и отправьте их:

git add .

git commit -m "Added push screenshot and project update"

git push

Подключение JavaScript к HTML

Откройте файл **index.html** и добавьте базовую **HTML-структуру**:

```
<!DOCTYPE html>
<html lang="ru-Ru">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Lab 19 - Работа с DOM в JS</title>
  </head>
  <body>
    <h1>Работа с DOM</h1>
    <script src="main.js"></script>
  </body>
</html>
```

Шаг 2. Что такое DOM

DOM (Document Object Model) — это объектная модель HTML-документа, представленная в виде древовидной структуры. Эта модель позволяет языкам программирования, таким как **JavaScript**, взаимодействовать со страницей: читать, изменять её содержимое и структуру, а также реагировать на действия пользователя.

Что можно делать с помощью JS, работая с DOM?

JavaScript, используя DOM, может:

- **Читать** элементы страницы и их содержимое.
- **Изменять** текст, HTML-разметку или атрибуты элементов.
- **Управлять** стилями (CSS) элементов.
- **Реагировать** на события (клики, наведение, ввод текста и др.), создавая интерактивность.

Практическая часть: изучение объекта document

1. Добавьте в файл **index.html**:

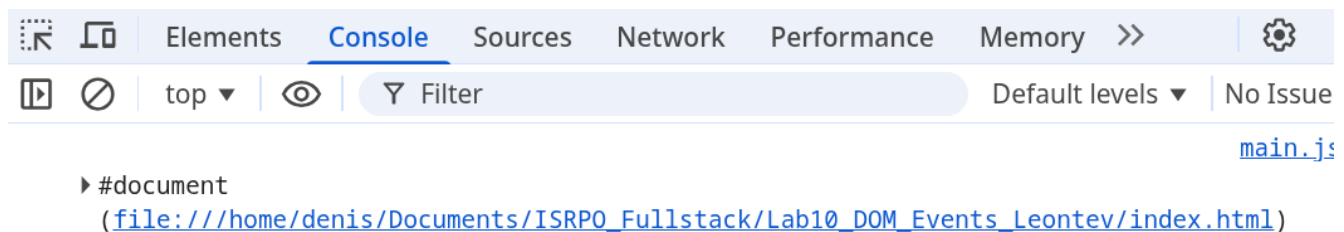
```
<body>
  <h1 id="title">Работа с DOM</h1>
  <p class="text">Это абзац</p>
```

2. Откройте файл **main.js**.

Для начала просто выведем в консоль глобальный объект document, который является "входной точкой" в DOM.

```
console.log(document);
```

Запустите **index.html** в **браузере** и откройте инструменты разработчика (**Консоль**):



Результат: Вы увидите развёрнутое дерево DOM-объектов, соответствующее структуре вашей HTML-страницы. Это и есть объектное представление документа, с которым будет работать JS.

Практическое задание

1. Сделайте скриншот **консоли браузера с выводом объекта document** и сохраните в папку **img** с именем:

step2_domStructureLab20_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 2: introduction to DOM"

git push

Шаг 3. Поиск и выбор элементов в DOM

Для работы с элементами на странице их сначала нужно найти. **JavaScript** предоставляет несколько методов для поиска элементов в **DOM-дереве**.

Добавьте следующий код в файл **main.js** после предыдущих команд:

1. Поиск элемента по его уникальному id:

```
const title = document.getElementById("title");
```

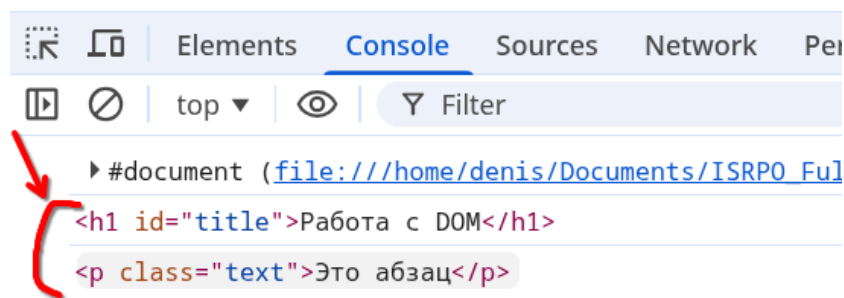
2. Универсальный поиск по CSS-селектору (находит первый подходящий элемент):

```
const text = document.querySelector(".text");
```

3. Выводим найденные элементы в консоль для проверки:

```
console.log(title);  
console.log(text);
```

Запустите **index.html** в **браузере** и откройте инструменты разработчика (**Консоль**):



Отличие методов

- **getElementById("id")** – находит элемент по его уникальному идентификатору (атрибут id). Самый быстрый и прямой метод, но работает только с id.
- **querySelector("селектор")** – мощный и универсальный метод. Находит первый элемент, соответствующий указанному CSS-селектору (можно искать по классу, тегу, атрибуту и их комбинациям). Примеры: **".class"**, **"#id"**, **"p"**, **"input[type='text']"**.

Практическое задание

1. Сделайте скриншот **консоли браузера с выводом** и сохраните в папку **img** с именем:

step3_domSelectionLab20_ФИО.png

2. Выполните следующие команды:

git add .

git commit -m "Step 3: selecting DOM elements"

git push

Шаг 4. Изменение содержимого и стилей элементов

После того как элемент найден и сохранён в переменную, мы можем:

- Менять его текстовое содержимое.
- Изменять CSS-стили напрямую через JavaScript.

Добавьте следующий код в файл **main.js**:

1. Изменяем текстовое содержимое заголовка:

```
title.textContent = "DOM изменён через JS!";
```

2. Меняем стили элемента через свойство style:

```
title.style.color = "blue";  
title.style.backgroundColor = "■ #f0f0f0";  
title.style.padding = "10px";
```

3. Меняем стили абзаца:

```
text.style.fontSize = "18px";  
text.style.fontStyle = "italic";  
text.style.color = "darkgreen";
```

Запустите **index.html** в браузере и посмотрите на изменения:

DOM изменён через JS!

Это абзац

Практическое задание

1. Сделайте скриншот **страницы с результатом работы** и сохраните в папку **img** с именем:

step4_domManipulationLab20_ФИО.png

2. Выполните следующие команды:

git add .

git commit -m "Step 4: modifying DOM content and styles"

git push

Шаг 5. Работа с событиями: обработчик клика

События (events) - это действия, происходящие на веб-странице (клик мыши, нажатие клавиши, загрузка страницы и др.). JavaScript позволяет "слушать" эти события и реагировать на них с помощью функций-обработчиков.

Добавьте кнопку в файл **index.html**:

```
<body>
  <h1 id="title">Работа с DOM</h1>
  <p class="text">Это абзац</p>
  <button id="btn">Нажми меня</button>
```

Добавьте следующий код в **main.js**.

1. Находим кнопку по id:

```
const btn = document.getElementById("btn");
```

2. Добавляем обработчик события "click":

```
btn.addEventListener("click", () => {
  // ...
});
```

3. Добавляем действие, которое выполнится при клике и изменяем стиль кнопки после клика:

```
btn.addEventListener("click", () => {
  alert("Кнопка нажата!");
  btn.style.backgroundColor = "#4CAF50";
  btn.style.color = "white";
});
```

Основные понятия:

- **addEventListener()** - метод для назначения обработчика события
- **"click"** - тип события (в данном случае - клик мыши)
- **Стрелочная функция () => { ... }** - функция-обработчик, которая выполнится при возникновении события

Практическое задание

1. Сделайте скриншот **страницы с результатом работы** и сохраните в папку **img** с именем:

step5_clickEventLab20_ФИО.png

2. Выполните следующие команды:

git add .

git commit -m "Step 5: click event handling"

git push

Шаг 6. Работа с элементами формы: input и события

Для создания интерактивных веб-страниц важно уметь:

- Получать данные от пользователя через элементы форм (например, `<input>`);
- Реагировать на действия пользователя с помощью событий;
- Динамически обновлять контент на странице на основе этих действий.

Обновите файл **index.html**, добавив форму ввода и элементы для взаимодействия:

```
<button id="btn">Нажми меня</button>
<div>
  <input type="text" id="nameInput" placeholder="Введите ваше имя" />
  + <button id="showName">Показать приветствие</button>
  <p id="output"></p>
</div>
<script src="main.js"></script>
```

Добавьте следующий код в файл **main.js**.

1. Находим все необходимые элементы:

```
const input = document.getElementById("nameInput");
const button = document.getElementById("showName");
const output = document.getElementById("output");
```

2. Добавляем обработчик события "click" на кнопку:

```
button.addEventListener("click", () => {
  . .
});
```

3. Внутри нашего обработчика события проверяем, не пустое ли поле ввода, и через блок `if` формируем и выводим приветствие:

```
button.addEventListener("click", () => {
  if (input.value.trim() !== "") {
    output.textContent = `Привет, ${input.value}!`;
    output.style.color = "green";
  }
});
```

4. Если поле пустое - показываем сообщение об ошибке:

```
button.addEventListener("click", () => {
  if (input.value.trim() !== "") { ...
} else {
  output.textContent = "Пожалуйста, введите имя!";
  output.style.color = "red";
}
});
```

Ключевые моменты:

- **addEventListener()** – метод для "подписки" на события элементов
- **input.value** – получение значения из поля ввода

Практическое задание

1. Сделайте скриншот **страницы с результатом работы** и сохраните в папку

img с именем:

step6_inputEventLab20_ФИО.png

2. Выполните следующие команды:

git add .

git commit -m "Step 6: input and button interaction"

git push

Шаг 7. Закрепление навыков работы с DOM

Расширим функциональность предыдущего кода:

1. Добавим вторую кнопку "Очистить"
2. При нажатии на неё:
 - Очищаем поле ввода;
 - Очищаем текст результата (#output).

Обновите **index.html**:


```
<button id="showName">Показать приветствие</button>
<button id="clearButton">Очистить</button>]+
<p id="output"></p>
```

Добавьте код в **main.js**:

1. Находим новую кнопку:

```
const clearButton = document.getElementById("clearButton");
```

2. Добавляем обработчик события для кнопки "Очистить":

```
clearButton.addEventListener("click", () => {
});
```

3. Внутри обработчика очищаем поле ввода и текст результата:

```
clearButton.addEventListener("click", () => {
+ {input.value = "";
+ {output.textContent = "";
});
```

4. Возвращаем стандартный цвет и фокусируемся обратно на поле ввода:

```
output.style.color = "";
input.focus();
```

focus() — устанавливает фокус на элемент (курсор в поле ввода).

Когда полезно:

- После очистки формы → вернуть курсор в поле
- При ошибке валидации → фокус на поле с ошибкой
- После добавления задачи → готовность к следующей

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:

step7_miniTaskLab20_ФИО.png

2. Выполните следующие команды:

git add .

git commit -m "Step 7: mini task with DOM events"

git push

Шаг 8. HTML-формы и получение данных

HTML-формы - основной способ сбора данных от пользователя на веб-страницах. JavaScript позволяет:

- Читать значения из полей формы;
- Предотвращать стандартную отправку формы на сервер (перезагрузку страницы) для обработки данных на стороне клиента;
- Обрабатывать данные перед отправкой или для динамического отображения.

Добавьте в **index.html** форму для сбора данных пользователя:

```
<form id="userForm">
  <input type="text" id="username" placeholder="Имя пользователя" />
  <input type="number" id="age" placeholder="Возраст" />
  <button type="submit">Отправить</button>
</form>
<p id="formResult"></p>
<script src="main.js"></script>
```

Почему `type="submit"`?

Кнопка с типом `submit` внутри `<form>`:

- Отправляет форму при клике
- Отправляет форму при нажатии Enter в любом поле
- Срабатывает событие `submit` у формы

Добавьте в **main.js** код для работы с формой.

1. Находим элементы формы:

```
const form = document.getElementById("userForm");
const usernameInput = document.getElementById("username");
const ageInput = document.getElementById("age");
const result = document.getElementById("formResult");
```

2. Добавляем обработчик события отправки формы:

```
form.addEventListener("submit", (event) => {
  // ...
});
```

3. Предотвращаем стандартную отправку формы (перезагрузку страницы):

```
form.addEventListener("submit", (event) => {
  + event.preventDefault();
  // ...
});
```

4. Получаем значения из полей:

```
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = usernameInput.value;
  const age = ageInput.value;
});
```

5. Выводим результат:

```
const age = ageInput.value;
result.textContent = `Имя: ${name}, Возраст: ${age}`;
});
```

Ключевые моменты:

- **event.preventDefault()** - отменяет стандартное поведение формы
- Событие **"submit"** срабатывает при отправке формы (клик на кнопку типа submit или нажатие Enter)
- Значения полей доступны через свойство **.value**

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:
step8_formInputLab20_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 8: basic HTML form handling"

git push

Шаг 9. Валидация данных формы

Валидация - процесс проверки корректности введенных пользователем данных.

Основные типы проверок:

- **Обязательные поля** - проверка на пустые значения;
- **Допустимый диапазон** - проверка числовых значений на соответствие ограничениям;
- **Формат данных** - проверка соответствия шаблону (email, телефон и т. д.);
- **Пользовательские сообщения** - информирование пользователя об ошибках понятным языком.

Модифицируйте обработчик формы для добавления валидации:

```
// trim() удаляет пробелы по краям
const name = usernameInput.value.trim();
// Преобразуем строку в число
const age = Number(ageInput.value);
```

1. Валидация имени:

```
if (name === "") {
    result.textContent = "Ошибка: Имя не может быть пустым!";
    result.style.color = "red";
    usernameInput.focus(); // Фокусируемся на поле с ошибкой
    return; // Прерываем выполнение функции
}
```

2. Валидация возраста:

```
if (isNaN(age) || age <= 0 || age > 120) {
    result.textContent = "Ошибка: Введите корректный возраст (от 1 до 120)!";
    result.style.color = "red";
    ageInput.focus();
    ageInput.value = ""; // Очищаем некорректное значение
    return;
}
```

3. Если все проверки пройдены - успешное сообщение:

```
result.textContent = `Данные сохранены: ${name}, ${age} лет`;
result.style.color = "green";
```

4. Очищаем форму после успешной отправки:

```
form.reset();
```

Улучшенная валидация:

- **trim()** - удаляет лишние пробелы в начале и конце строки
- **isNaN()** - проверяет, является ли значение "не числом"
- **focus()** - устанавливает фокус на поле с ошибкой для удобства пользователя
- **return** - прерывает выполнение функции при обнаружении ошибки
- **form.reset()** - очищает все поля формы после успешной отправки

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:
step9_formValidationLab20_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 9: form validation in JavaScript"

git push

Шаг 10. Динамическое создание элементов

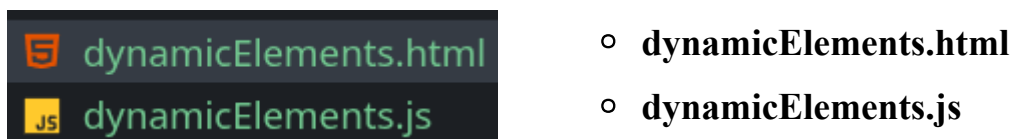
JavaScript позволяет не только изменять существующие элементы, но и создавать новые прямо во время выполнения скрипта. Это важно для:

- Динамического добавления контента без перезагрузки страницы
- Создания списков, таблиц и других структур на основе данных
- Реализации интерактивных интерфейсов (комментарии, задачи, товары и т.д.)

Основные методы:

- **document.createElement(tagName)** - создаёт новый HTML-элемент
- **element.appendChild(child)** - добавляет элемент как последний дочерний
- **element.innerHTML** - устанавливает или получает HTML-содержимое элемента

Создайте **два** новых файла в проекте:



Добавьте основную разметку в файл **dynamicElements.html**, вставьте контейнер для списка, кнопку добавления и ссылку на новый скрипт:

```
<body>
<div class="user-list-container">
  <h3>Список пользователей</h3>
  <button id="addUserBtn">Добавить пользователя</button>
  <ul id="usersList"></ul>
</div>
<script src="dynamicElements.js"></script>
</body>
```

В **dynamicElements.js** напомним следующий код.

1. Находим необходимые элементы:

```
const addUserBtn = document.getElementById("addUserBtn");
const usersList = document.getElementById("usersList");
```

2. Добавляем счётчик для уникальных ID (в реальном проекте используются более сложные системы):

```
let userCounter = 1;
```

3. Обработчик для кнопки "Добавить пользователя":

```
addUserBtn.addEventListener("click", () => {  
  
});
```

4. Внутри обработчика создаём новый элемент списка :

```
addUserBtn.addEventListener("click", () => {  
+const listItem = document.createElement("li");
```

5. Внутри обработчика создаём текстовый узел с содержимым:

```
const userText = document.createTextNode(`Пользователь #${userCounter}`);
```

6. Внутри обработчика создаём кнопку удаления:

```
const deleteBtn = document.createElement("button");  
deleteBtn.textContent = "Удалить";  
deleteBtn.className = "delete-btn";  
deleteBtn.style.marginLeft = "10px";  
deleteBtn.style.fontSize = "12px";
```

7. Внутри обработчика собираем элемент:

```
listItem.appendChild(userText);  
listItem.appendChild(deleteBtn);  
listItem.id = `user-${userCounter}`;
```

8. Внутри обработчика добавляем элемент в список:

```
usersList.appendChild(listItem);
```

9. Внутри обработчика увеличиваем счётчик и обновляем статистику:

```
userCounter++;  
updateUserStats();
```

10. Создаём новую функцию для удаления пользователей (делегирование событий)

```
usersList.addEventListener("click", (event) => {
  if (event.target.classList.contains("delete-btn")) {
    const listItem = event.target.parentElement;
    listItem.remove();
    updateUserStats();
  }
});
```

Важно: Делегирование событий

Проблема: Когда элементы создаются динамически, нельзя заранее "повесить" на них обработчики.

Данный код не работает:

```
const deleteButtons = document.querySelectorAll(".delete-btn");
deleteButtons.forEach(btn => {
  btn.addEventListener("click", () => {
    // Удаление
  });
});
```

Решение: Делегирование

Вешаем обработчик на **родителя** (который существует всегда):

Событие "всплывает" от кнопки к родителю.

Преимущества:

- Работает с динамически созданными элементами
- Один обработчик вместо сотен
- Меньше потребление памяти

Сравнение с C#:

В WinForms/WPF вы "вешали" обработчики на каждый элемент:

```
button1.Click += HandleClick;
```

```
button2.Click += HandleClick;
```

В JavaScript можно **один раз** на родителя:

```
parent.addEventListener("click", handleClick);
```

10. Создаём новую функцию обновления статистики:


```
function updateUserStats() {
  const userCount = userList.children.length;
  const statsElement =
    document.querySelector(".user-stats") || createStatsElement();
  statsElement.textContent = `Всего пользователей: ${userCount}`;
}
```

11. Функция создания элемента статистики:

```
function createStatsElement() {
  const stats = document.createElement("div");
  stats.className = "user-stats";
  stats.style.marginTop = "10px";
  stats.style.fontWeight = "bold";
  stats.style.color = "■ #333";
  userList.parentElement.appendChild(stats);
  return stats;
}
```

Запустите в браузере **dynamicElements.html** и проверьте работу:

Список пользователей

Добавить пользователя

- Пользователь #1
- Пользователь #2
- Пользователь #3
- Пользователь #4

Всего пользователей: 4

Практическое задание

1. Сделайте скриншот страницы с результатом работы в папку **img** с именем:

step10_dynamicElementsLab20_FIO.png

2. Выполните следующие команды:

git add .

git commit -m "Step 10: dynamic DOM element creation"

git push

Самостоятельное задание

Цель задания: Создать подробное и структурированное описание лабораторной работы №20 в файле README.md, используя Markdown.

Что должно быть в README.md

1. Заголовок

2. Основная информация

****ФИО:** Иванова Иван Иванович**

****Группа:** ИСП-231**

****Дата:** 18.02.2026**

3. Краткое описание работы

Описание (что изучили)

4. Структура проекта

Структура проекта

Сравнение с C# (WinForms/WPF)

DOM в JS = Controls в C# WinForms

Концепция	C# (WinForms)	JavaScript (DOM)
Найти элемент	Button myButton = this.Controls["myButton"]	const myButton = document.getElementById("myButton")
Изменить текст	label1.Text = "Новый текст"	label.textContent = "Новый текст"
Добавить обработчик	button1.Click += HandleClick	button.addEventListener("click", handleClick)
Создать элемент	Button btn = new Button()	const btn = document.createElement("button")
Добавить в контейнер	panel1.Controls.Add(btn)	panel.appendChild(btn)
Скрыть элемент	button1.Visible = false	button.style.display = "none"

Events в JS = События в C#

C# (WinForms):

```
private void Button1_Click(object sender, EventArgs e)
{
    label1.Text = "Нажато!";
}
```

JavaScript (DOM):

```
button.addEventListener("click", (e) => {
    label.textContent = "Нажато!";
});
```

Валидация формы

C# (WinForms):

```
private void SubmitButton_Click(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(nameTextBox.Text))
    {
        MessageBox.Show("Введите имя!");
        return;
    }
    // ...
}
```

JavaScript (DOM):

```
form.addEventListener("submit", (e) => {
    e.preventDefault();

    if (!nameInput.value.trim()) {
        alert("Введите имя!");
        return;
    }
    // ...
});
```

Главные отличия:

Аспект	C# WinForms	JavaScript DOM
Компиляция	Да	Нет (интерпретация)
Типизация	Строгая	Динамическая
Ошибки	Во время компиляции	Во время выполнения
UI-дизайнер	Есть (визуальный)	Нет (только код)
Платформа	Windows	Любой браузер

Преимущества DOM перед WinForms:

- Кроссплатформенность (работает везде)
- Не нужна установка приложения
- Легко обновлять (обновил файлы — всё работает)
- Современные UI (CSS, анимации)

Преимущества WinForms перед DOM:

- Визуальный дизайнер (drag & drop)

- Ошибки на этапе компиляции
- Интеграция с Windows API
- Более понятная структура

Вывод: DOM и WinForms решают схожие задачи, но разными подходами.

Финальный коммит (после всех шагов):

git add .

git commit -m "Completed Lab 19: JavaScript DOM"

git push